Remove comma elision in variadic function declarations

Document number:	P0281R0
Date:	2016-01-23
Project:	Programming Language C++, Evolution Working Group

Reply-to:	James Touton <bekenn@gmail.com>

Table of Contents

Introduction

C++ currently allows variadic functions to be declared without requiring a comma before the ellipsis marking the variadic arguments to the function. This rarely-used feature leads to confusing constructs, is an impediment to future language design, and provides no value. It should be removed from the language.

The following declarations are all legal C++:

```
int f(...);                 // 1. function taking only variadic arguments
int g(int i, ...);          // 2. function taking explicit and variadic arguments, with comma
int h(int i ...);           // 3. function taking explicit and variadic arguments, without comma
```

The first declaration provides no portable way to get at the function arguments and is mainly useful for metaprogramming. The second declaration is by far the most common form of variadic function declaration in use. The third declaration is similar to the second, but omits the comma separating the last explicit parameter declaration from the ellipsis.

This proposal aims to make the third declaration illegal.

Motivation and Scope

Ambiguity

Comma elision gives rise to an ambiguity called out in the standard (§8.3.5 [decl.fct] paragraph 17):

> There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.

The following declarations are both legal:

```
template <class... T> void f(T...); // declares a function template with a parameter pack
template <class T> void f(T...);    // declares a template for a variadic function
```

The meaning of the ellipsis in the function parameter list is different in these two declarations:

- In the first declaration, the ellipsis establishes a function parameter pack that takes its type arguments from the corresponding template parameter pack `T`; the resulting function can take any number of arguments, all of which must be convertible to a corresponding member of `T`. In the function body (defined elsewhere), the parameter pack may be expanded with the pack expansion operator, its size queried using the `sizeof...` operator, and can participate in fold expressions.
- In the second declaration, the ellipsis establishes that the function `f` takes a variable number of arguments of any type, following the first argument, which must be convertible to `T`. The resulting arguments are promoted before being passed to the function, and type information is lost; the function must make assumptions about the argument types and can only access the arguments using the machinery from the `<cstdarg>` header.

Future language design

The following function template is currently not legal C++:

```
template <>
bool multi_and(bool... args)
{
    return true && ... && args;
}
```

This code is currently invalid, but illustrates a reasonable future extension to C++. In this hypothetical extension, `args` is a function parameter pack with no corresponding template parameter pack. (`multi_and` is still a template, because information about the size of `args` is required at compile time and is different for each instantiation.) The declaration of `args` is not a pack expansion; the size of the parameter pack is deduced from the number of arguments passed to the function.

The corresponding forward declaration with an abstract parameter declarator runs into the same ambiguity mentioned earlier:

```
template <> bool multi_and(bool...);    // parameter pack or varargs?
```

This declaration is also currently invalid due to the empty template parameter list, so it may seem obvious and sufficient to disambiguate in favor of a parameter pack; however, incorporating abbreviated templates from Concepts, it becomes possible to drop the template parameter list entirely while still declaring a template:

```
bool multi_and(bool... args);   // ok; template declaration with parameter pack
bool multi_and(bool...);        // ambiguous; template, or variadic function?
```

With abbreviated template declarations, the form with the abstract parameter declarator matches existing syntax for declaring a variadic function. The extension thus either introduces a subtle inconsistency for Scott Meyers to write about, or it silently changes the meaning of currently valid code. Comma elision is thus a barrier to entry for this extension.

Compatibility

Variadic function declarations using comma elision cannot appear in a header file that is to be used by a C program. No version of the C standard going back at least as far as ISO/IEC 9899:1990 has included support for comma elision.

This is the production for *parameter-type-list* from the most recent C standard, ISO/IEC 9899:2011 §6.7.6:

> *parameter-type-list*:
> *parameter-list*
> *parameter-list* , ...

The C language requires at least one fixed parameter to be declared preceding an ellipsis in a variadic function declaration, and further requires a separating comma. C language compatibility is therefore not adversely affected by the removal of comma elision in C++ (and is in fact slightly enhanced).

Impact On the Standard

This proposal removes an existing feature and breaks existing code.

It is the author's belief that this change would hardly be noticed throughout the industry. C++ provides a number of compelling alternatives to C-style variadic functions, including function overloading, initializer lists, and variadic function templates; the use of C-style variadic functions is discouraged due to the lack of information available to the function regarding the number and types of arguments. In C++, C-style variadic functions are rarely used except when interoperating with C code. In order to be compatible with C code, the declaration of a C-style variadic function must conform to the stricter C language requirements that forbid the use of comma elision.

Existing code that makes use of comma elision can easily and trivially be updated to conform to stricter rules by inserting a comma before the ellipsis. Existing tools and compilers that already recognize comma elision (as all currently conforming implementations must) can be updated to provide "fix-it" hints or even automatic transformations of existing code.

Although this is a breaking change, it is not out of line with earlier changes to the C++ standard. When user-defined literals were adopted, formerly-valid lexical sequences became invalid: An identifier that immediately follows a string literal with no intervening white space could no longer be interpreted as a macro name.

```
#define SUFFIX "cadabra"
const char* magic_word = "abra"SUFFIX;  // ok in C++98, error in C++11
```

The fix for this change was to add separating white space between the string literal and the macro name. This was not a significant burden for developers.

Wording

All modifications are presented relative to N4567.

Modify §8.3.5 [dcl.fct] paragraph 3:

> A type of either form is a *function type*.
>
> *parameter-declaration-clause*:
> *parameter-declaration-list*$_{opt}$ ~~, ...~~$_{opt}$
> , ...
> *parameter-declaration-list* , ...
> *parameter-declaration-list*:
> *parameter-declaration*
> *parameter-declaration-list* , *parameter-declaration*
> *parameter-declaration*:
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq declarator*
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq declarator* = *initializer-clause*
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$ = *initializer-clause*
>
> The optional *attribute-specifier-seq* in a *parameter-declaration* appertains to the parameter.

Modify §8.3.5 [dcl.fct] paragraph 4:

> The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [*Note:* the *parameter-declaration-clause* is used to convert the arguments specified on the function call; see 5.2.2. —*end note*] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type `void` is equivalent to an empty parameter list. Except for this special case, a parameter shall not have type *cv* `void`. If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack (14.5.3), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. ~~Where syntactically correct and where "..." is not part of an *abstract-declarator*, ", ..." is synonymous with "...".~~ [*Example:* the declaration
>
> ```
> int printf(const char*, ...);
> ```
>
> declares a function that can be called with varying numbers and types of arguments.
>
> ```
> printf("hello world");
> printf("a=%d b=%d", a, b);
> ```
>
> However, the first argument must be of a type that can be converted to a `const char*` —*end example*] [*Note:* The standard header `<cstdarg>` contains a mechanism for accessing arguments passed using the ellipsis (see 5.2.2 and 18.10). —*end note*]

Delete §8.3.5 [dcl.fct] paragraph 17:

> ~~There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.~~